OAVS TGT COMPUTER SCIENCE EXAMINATION

Comprehensive MCQ Question Bank

TOPIC: FILE — Concept · Organisation · Access Techniques · Handling Functions

Total Questions	270
Topics Covered	Concept of File | Indexed Org. | Line Sequential Org. | Hashed Org. Sequential Access | Indexed Access | Hashed Access Sorting | Merging | Indexing | Updating
Question Types	Basic · Conceptual · Application · Assertion-Reason Match-the-Following · True/False · Statement · Comparison · Scenario
Exam Standard	OAVS TGT Computer Science Teacher Examination
Format	Each Q: 4 options (A-D) + Correct Answer + Short Explanation
Answer Distribution	A: ~27% | B: ~26% | C: ~24% | D: ~23% (Randomised)
Restriction	STRICTLY limited to OAVS TGT CS syllabus screenshot topics only

Zero out-of-syllabus questions | No duplicate questions | Explanation after every question | Answers randomised across A/B/C/D

TABLE OF CONTENTS

#	Section	Question Range	Count
1	CONCEPT OF FILE	Q1–Q50	50
2	FILE ORGANISATION TECHNIQUES	Q51–Q110	60
3	FILE ACCESSING TECHNIQUES	Q111–Q150	40
4	FILE HANDLING FUNCTIONS — SORTING, MERGING, INDEXING, UPDATING	Q151–Q230	80
5	COMPREHENSIVE, COMPARISON, ASSERTION-REASON & SCENARIO Qs	Q231–Q270	40

ANSWER DISTRIBUTION (after randomisation): Option A: 74 questions (27.4%) | Option B: 69 questions (25.6%) | Option C: 64 questions (23.7%) | Option D: 63 questions (23.3%)

INSTRUCTIONS: 1) Each question has FOUR options — only ONE is correct. 2) Correct answer shown in green after options. 3) A short explanation follows every question. 4) All questions are strictly within the OAVS TGT CS File syllabus only. 5) No question is repeated. 6) Answer options have been randomised — correct answer is equally likely to be A, B, C, or D.

SECTION 1: CONCEPT OF FILE

Q1. A 'file' in data storage is BEST defined as:

A) A temporary variable in a program
B) A named collection of related records stored on secondary storage
C) A pointer to another data structure
D) A single record kept in RAM

■ **Correct Answer: (B)**

Explanation: *A file is a named, persistent collection of related records stored on secondary storage (disk/tape). Unlike RAM variables, files survive program termination.*

Q2. The SMALLEST meaningful unit of data in a file is:

A) Block
B) Field
C) Record
D) File

■ **Correct Answer: (B)**

Explanation: *A field (attribute) is the smallest unit — it holds one piece of information, e.g., Employee_Name or Salary. Multiple fields combine to form a record.*

Q3. A 'record' in a file is BEST described as:

A) A group of related fields describing one entity
B) An index pointer
C) A complete collection of files
D) A single character value

■ **Correct Answer: (A)**

Explanation: *A record groups all fields about one entity (e.g., all data about one employee). Multiple records form a file.*

Q4. The correct data hierarchy from SMALLEST to LARGEST unit is:

A) Record → Field → File
B) Field → Record → File
C) File → Record → Field
D) Field → File → Record

■ **Correct Answer: (B)**

Explanation: *Data hierarchy: Bit → Character → Field → Record → File. Among given options, Field → Record → File is correct.*

Q5. A file that holds PERMANENT reference data about an organisation is called a:

A) Work file
B) Master file
C) Backup file
D) Transaction file

■ **Correct Answer: (B)**

Explanation: *A master file contains permanent, up-to-date reference data (e.g., Employee Master File). It is updated periodically using a transaction file.*

Q6. A file that holds data about RECENT EVENTS (e.g., today's sales) is called a:

A) Transaction file
B) Index file
C) Backup file
D) Master file

■ **Correct Answer: (A)**

Explanation: *A transaction (detail/change) file stores recent event data that will be used to update the master file in a batch update cycle.*

Q7. Which type of file is created TEMPORARILY during processing and then discarded?

A) Work file
B) Backup file
C) Master file
D) Transaction file

■ **Correct Answer: (A)**

Explanation: *A work file (scratch file) holds intermediate results during processing and is not retained after the job completes.*

Q8. A backup file is primarily used to:

A) Recover data after a system failure or disaster
B) Index the master file
C) Speed up record retrieval
D) Sort records in key order

■ **Correct Answer: (A)**

Explanation: *Backup files are copies of master files kept for recovery. If the current master is damaged, it can be restored from the backup.*

Q9. Which of the following is a characteristic of files stored on SECONDARY STORAGE?

A) Data is lost when power is switched off
B) Access is always faster than RAM
C) Data persists after the program ends
D) Cannot be shared between programs

■ **Correct Answer: (C)**

Explanation: *Secondary storage is non-volatile. Files persist after program execution ends — unlike RAM, which loses data on power-off.*

Q10. A 'key field' in a file is used to:

A) Mark the end of the file
B) Store the hash value of a record
C) Store the file name on disk
D) Uniquely identify each record in the file

■ **Correct Answer: (D)**

Explanation: *A key field (primary key) holds a unique value for each record, enabling precise identification and retrieval.*

Q11. EOF stands for:

A) End of File
B) End of Format
C) Error of Function
D) Entry of Field

■ **Correct Answer: (A)**

Explanation: *EOF (End of File) is a marker signalling there are no more records. Programs check for EOF to exit reading loops.*

Q12. Which is an example of a PRIMARY KEY in a student file?

A) Date of Birth
B) City of Residence
C) Student Roll Number
D) Student Name

■ **Correct Answer: (C)**

Explanation: *Roll number uniquely identifies every student with no duplicates, making it the primary key. Name and city can repeat.*

Q13. FIXED-LENGTH records in a file mean:

A) Records use a hash function for storage
B) Every record occupies exactly the same number of bytes
C) Each record has a different size
D) Records are always sorted

■ **Correct Answer: (B)**

Explanation: *Fixed-length records allow easy address calculation: address of record N = (N-1) × record_length. No scanning is needed to locate record N.*

Q14. VARIABLE-LENGTH records are used when:

A) The file is always sorted
B) Hashing is used for all access
C) Some fields vary in length across different records
D) All fields have the same size for every record

■ **Correct Answer: (C)**

Explanation: *When fields like 'address' or 'remarks' differ in length per record, variable-length records avoid wasting space with padding.*

Q15. The BLOCKING FACTOR of a file refers to:

A) Number of files in a directory
B) Number of logical records stored in one physical disk block
C) Number of hash buckets
D) Number of index levels

■ **Correct Answer: (B)**

Explanation: *Blocking factor = logical records per physical block. Higher blocking factor → more records per I/O → fewer physical reads needed.*

Q16. A 'LOGICAL RECORD' is:

A) A hash bucket
B) The actual data block read from disk
C) A disk block containing multiple records
D) The record as seen and used by the application program

■ **Correct Answer: (D)**

Explanation: *A logical record is the programmer's view (fields grouped together). A physical record (block) may contain several logical records.*

Q17. The 'ACTIVITY RATIO' of a file processing run is:

A) Proportion of records accessed in one processing run
B) Number of hash collisions
C) Number of indexes on the file
D) Number of fields per record

■ **Correct Answer: (A)**

Explanation: *Activity ratio = records accessed ÷ total records. High activity ratio (many records processed) favours sequential access.*

Q18. File 'VOLATILITY' refers to:

A) Frequency of insertions and deletions relative to file size
B) Number of indexes on the file
C) Speed of disk access
D) Encryption level of the file

■ **Correct Answer: (A)**

Explanation: *Volatility measures how often a file changes. Highly volatile files have frequent additions/deletions, affecting the best choice of organisation.*

Q19. A 'REPORT FILE' is used to:

A) Hash records for direct storage
B) Index records for fast access
C) Produce formatted output for printing or display
D) Store permanent employee data

■ **Correct Answer: (C)**

Explanation: *A report file contains formatted data ready for printing or display. It is an output file, not used for updating master data.*

Q20. Which of the following is NOT a basic file operation?

A) Create
B) Read
C) Compile
D) Write

■ **Correct Answer: (C)**

Explanation: *Basic file operations: Create, Open, Read, Write, Update, Delete, Close. Compile is a programming-language operation, not a file operation.*

Q21. The FILE POINTER in a program indicates:

A) The name of the file on disk
B) The hash value of the last record
C) The physical sector number on disk
D) The current read/write position within the file

■ **Correct Answer: (D)**

Explanation: *The file pointer tracks where the next read or write will occur. After each operation, it advances automatically to the next position.*

Q22. A 'SECONDARY KEY' in a file is:

A) Always unique for every record
B) Stored only inside the index
C) The same as the hash key
D) A non-primary field used for searching (may not be unique)

■ **Correct Answer: (D)**

Explanation: *A secondary key allows searching on non-primary attributes (e.g., find all employees in a department). It may not be unique across records.*

Q23. Which statement about files is FALSE?

A) Files persist after program execution ends
B) Files contain records made up of fields
C) Files are always stored in RAM
D) Files can be shared between programs

■ **Correct Answer: (C)**

Explanation: *Files are stored on secondary storage (disk/tape), NOT in RAM. RAM is volatile primary memory.*

Q24. The LOGICAL END of a sequential file is marked by:

A) The EOF marker
B) An empty index
C) A null field value
D) A hash collision indicator

■ **Correct Answer: (A)**

Explanation: *The EOF marker signals no more records exist. Programs use it to exit reading loops cleanly.*

Q25. Which file type is HUMAN-READABLE?

A) Text file
B) Binary object file
C) Compiled executable
D) Encrypted binary file

■ **Correct Answer: (A)**

Explanation: *Text files store data as printable ASCII/Unicode characters and can be opened in any text editor. Binary files are not directly human-readable.*

Q26. A BINARY FILE stores data:

A) In application-specific binary format, not readable as plain text
B) Using newline delimiters only
C) In plain ASCII text only
D) Always in sorted key order

■ **Correct Answer: (A)**

Explanation: *Binary files store raw binary values (e.g., integers as 4-byte representations). More compact than text but not human-readable.*

Q27. The HEADER of a file typically stores:

A) Index pointers only
B) Hash values of all records
C) Metadata about the file (record count, date, version, format)
D) All data records of the file

■ **Correct Answer: (C)**

Explanation: *A file header contains metadata — information about the file itself — allowing programs to correctly interpret the file before reading data.*

Q28. True/False: A file can have BOTH a primary key field AND a secondary key field.

A) True — one primary key and one or more secondary keys can coexist
B) True only for hashed files
C) False — only one key field is allowed
D) True only for indexed files

■ **Correct Answer: (A)**

Explanation: *Yes. The primary key uniquely identifies records; secondary keys allow searching on other attributes. Both can coexist in the same file.*

Q29. A 'FLAT FILE' is BEST described as:

A) A file with nested record structures
B) A simple single-table file with no relationships to other files
C) A file stored in a 3-dimensional structure
D) A file that always uses hashing

■ **Correct Answer: (B)**

Explanation: *A flat file stores data in a simple two-dimensional structure (rows = records, columns = fields) with no links or relationships to other files.*

Q30. A 'COMPOUND KEY' (composite key) is used when:

A) A single field is sufficient for unique identification
B) No single field is unique but a combination of fields is
C) The file uses hashing for storage
D) The file is always sorted on one field

■ **Correct Answer: (B)**

Explanation: *When no single field uniquely identifies a record, two or more fields are combined (e.g., Course_Code + Student_ID) to form a composite key.*

Q31. The OPEN operation on a file in READ mode:

A) Sorts all records in the file
B) Deletes all existing records
C) Hashes all records in the file
D) Positions the file pointer at the beginning for reading

■ **Correct Answer: (D)**

Explanation: *Opening a file for input (read) sets up system buffers and positions the pointer at the first record, ready for reading.*

Q32. 'FILE INTEGRITY' BEST means:

A) Speed of file access
B) Number of indexes on the file
C) Number of records in the file
D) Accuracy, completeness, and consistency of stored data

■ **Correct Answer: (D)**

Explanation: *File integrity ensures stored data is correct and consistent. It is maintained through validation checks, backup strategies, and controlled updates.*

Q33. A 'LOG FILE' records:

A) Permanent master data
B) All events and transactions for audit and recovery purposes
C) Hash values of all records
D) Index entries only

■ **Correct Answer: (B)**

Explanation: *Log files capture a complete history of all operations on a system, enabling audit trails and recovery by replaying logged transactions after a failure.*

Q34. Which of the following is an example of a WORK FILE?

A) Daily sales transaction file
B) Backup copy of the master file
C) Employee master file
D) Temporary sorted run file created during external merge sort

■ **Correct Answer: (D)**

Explanation: *During external merge sort, temporary sorted run files are created on disk. These are work files — used only during processing then deleted.*

Q35. The BLOCKING FACTOR improves file processing efficiency by:

A) Reducing the number of physical I/O operations per logical record processed
B) Building an index automatically
C) Increasing hash collisions
D) Sorting records automatically

■ **Correct Answer: (A)**

Explanation: *Higher blocking factor = more logical records per physical block = fewer physical reads. This significantly reduces disk I/O time, the main bottleneck.*

Q36. Which characteristic of a file measures the PROPORTION OF RECORDS THAT CHANGE in a given period?

A) Volatility
B) Blocking factor
C) Load factor
D) Activity ratio

■ **Correct Answer: (A)**

Explanation: *Volatility measures the rate of change — how many records are inserted or deleted relative to the total file size over a period of time.*

Q37. The CLOSE operation on a file:

A) Flushes write buffers to disk and releases the file from the program
B) Physically deletes the file from disk
C) Sorts all records in the file
D) Rehashes all records

■ **Correct Answer: (A)**

Explanation: *Closing a file flushes any data remaining in write buffers to disk and releases system resources (file handles) back to the operating system.*

Q38. A 'SCRATCH FILE' is another name for a:

A) Master file
B) Work/Temporary file
C) Transaction file
D) Backup file

■ **Correct Answer: (B)**

Explanation: *Scratch file = work file = temporary file. Created during processing for intermediate results and not retained after the job finishes.*

Q39. Which statement about a MASTER FILE is TRUE?

A) It is recreated entirely every day from scratch
B) It holds only today's transactions
C) It is always stored in RAM for fast access
D) It holds permanent, up-to-date reference data updated periodically

■ **Correct Answer: (D)**

Explanation: *The master file is the main long-term data store. It is updated at scheduled intervals using transaction files and is retained permanently.*

Q40. The GRANDFATHER-FATHER-SON backup concept keeps how many generations of master files?

A) Five
B) Three
C) Four
D) Two

■ **Correct Answer: (B)**

Explanation: *Three generations: son (current/new master), father (previous master), grandfather (one before that). If the son is corrupted, the father + latest transactions recreate it.*

Q41. In the grandfather-father-son scheme, the 'SON' file is:

A) The index file
B) The transaction file for the current run
C) The oldest backup copy
D) The most recently created (current) master file

■ **Correct Answer: (D)**

Explanation: *The son is the latest, most current version of the master file — produced from the father file plus the latest transaction file.*

Q42. Which of the following is NOT a recognised category of data file?

A) Compiled executable file
B) Transaction file
C) Master file
D) Backup file

■ **Correct Answer: (A)**

Explanation: *Compiled executable file is a programming concept (machine code), not a data file category. Master, transaction, and backup are recognised data file types.*

Q43. A file's 'PORTABILITY' means:

A) It is always encrypted for security
B) It can be used on different hardware/software platforms without conversion
C) It is automatically sorted whenever opened
D) It can be moved between folders on the same disk

■ **Correct Answer: (B)**

Explanation: *A portable file format (e.g., CSV, plain text) can be read by different programs on different systems, improving interoperability.*

Q44. The process of reading EVERY RECORD from the beginning to the end of a file exactly once is called:

A) Indexed access
B) Random access
C) Sequential scan / traversal
D) Hashed retrieval

■ **Correct Answer: (C)**

Explanation: *A sequential scan reads all records in stored order from first to last, processing each exactly once. Used for reports and batch updates.*

Q45. True/False: A KEY FIELD must always be a numeric data type.

A) False — a key field can be alphabetic, alphanumeric, or numeric
B) True
C) True only in indexed files
D) True only in hashed files

■ **Correct Answer: (A)**

Explanation: *Key fields can be any data type — Employee_ID (numeric), ISBN (alphanumeric), Product_Code (alphabetic). The requirement is uniqueness, not data type.*

Q46. Which of the following is an ADVANTAGE of using files for data storage?

A) Data persists between program executions and can be shared
B) Data is lost when power is switched off
C) Access is always faster than RAM
D) Data can be accessed by only one program at a time

■ **Correct Answer: (A)**

Explanation: *Files on secondary storage persist after programs end and can be shared between multiple programs — ideal for long-term data storage.*

Q47. A file that stores data in NON-HUMAN-READABLE binary format is called a:

A) Binary file
B) Log file
C) Configuration file
D) Text file

■ **Correct Answer: (A)**

Explanation: *Binary files store raw binary values and cannot be opened meaningfully in a text editor. They are compact and efficient for machine processing.*

Q48. The DELETE operation on a file:

A) Opens the file for reading
B) Removes the file from storage permanently
C) Closes the file
D) Sorts the file

■ **Correct Answer: (B)**

Explanation: *The DELETE operation removes the file's directory entry and marks its storage space as free. The data cannot be recovered through normal file operations.*

Q49. A file EXTENSION (e.g., .txt, .dat, .csv) typically indicates:

A) The name of the file owner
B) The file's physical size on disk
C) The file's type or format
D) The date and time of creation

■ **Correct Answer: (C)**

Explanation: *File extensions indicate format: .txt = text, .dat = data file, .csv = comma-separated values. The OS uses this to associate files with appropriate programs.*

Q60. In a FIXED-LENGTH record file, the address of record N can be calculated as:

A) (N-1) × record_length (from file start)
B) N + record_length
C) N / record_length
D) N × record_length

■ **Correct Answer: (A)**

Explanation: *For fixed-length records, each record occupies exactly the same space. Record N starts at byte offset (N-1) × record_length from the file beginning.*

SECTION 2: FILE ORGANISATION TECHNIQUES

Q61. In INDEXED file organisation, records are located using:

A) Line numbers in a text file
B) A hash function applied to the key
C) An index table that maps keys to disk addresses
D) Sequential scanning from the beginning

■ **Correct Answer: (C)**

Explanation: *An index stores (key, address) pairs. To access a record, the sorted index is searched (binary search) and the address retrieved is used to fetch the record directly.*

Q62. ISAM stands for:

A) Indexed Sequential Access Method
B) Integrated Sequential Access Module
C) Internal Sorting and Merging
D) Indexed Sequential Accessing Mode

■ **Correct Answer: (A)**

Explanation: *ISAM (Indexed Sequential Access Method) is a classic indexed file organisation with a multi-level index (master, cylinder, track) over physically sorted records.*

Q63. In ISAM, records are physically stored:

A) In random order across the disk
B) By their computed hash address
C) In the order they were inserted
D) In sorted order by the key field

■ **Correct Answer: (D)**

Explanation: *ISAM stores records in sorted key order on disk. The index allows both direct (random) access to individual records and sequential access through all records in key order.*

Q64. A DENSE INDEX contains:

A) One entry for every data block on disk
B) Only entries for logically deleted records
C) Entries for secondary keys only
D) One entry for every record in the file

■ **Correct Answer: (D)**

Explanation: *A dense index has one index entry per data record. This allows any record to be found directly from the index without reading a whole block sequentially.*

Q65. A SPARSE INDEX contains:

A) Entries for hash collision records
B) One entry for every record
C) Only entries for overflow records
D) One entry for every data block (not every record)

■ **Correct Answer: (D)**

Explanation: *A sparse index has one entry per block. It is smaller than a dense index but may require a brief sequential scan within a block after finding the block via the index.*

Q66. The PRIMARY ADVANTAGE of indexed file organisation is:

A) No extra storage needed for the index structure
B) Always faster than hashing for single-record lookup
C) Supports both sequential AND direct (random) access
D) Does not require a key field

■ **Correct Answer: (C)**

Explanation: *Indexed files support sequential processing (reading records in key order) AND direct access (using the index to jump to a specific record).*

Q67. A DISADVANTAGE of indexed file organisation is:

A) Cannot support random access
B) Cannot support sequential access
C) Requires additional storage space for the index and maintenance on every update
D) Records cannot be updated

■ **Correct Answer: (C)**

Explanation: *Every indexed file carries the overhead of the index structure (extra space). The index must also be kept current on every insert, delete, or key-change operation.*

Q68. In ISAM, the OVERFLOW AREA stores:

A) New records that cannot fit in the primary (full) data area
B) Index entries only
C) Hash collision records
D) Logically deleted records

■ **Correct Answer: (A)**

Explanation: *When a primary cylinder/track is full and new records arrive, they go to the overflow area. Over time, long overflow chains degrade retrieval performance.*

Q69. VSAM stands for:

A) Virtual Storage Access Method
B) Volume Storage and Merging
C) Variable Sorted Access Method
D) Virtual Sequential Access Memory

■ **Correct Answer: (A)**

Explanation: *VSAM (Virtual Storage Access Method) is IBM's improved indexed sequential organisation, offering KSDS (key-sequenced), ESDS (entry-sequenced), and RRDS (relative-record) datasets.*

Q70. A SECONDARY INDEX in a file is based on:

A) The physical line number
B) The primary key field
C) The computed hash function
D) A non-primary key field (any other attribute)

■ **Correct Answer: (D)**

Explanation: *A secondary index enables fast lookup by non-primary attributes (e.g., all employees in department X). Unlike the primary index, secondary keys may not be unique.*

Q71. A CLUSTERED INDEX means:

A) The physical storage order of records matches the index key order
B) Records are stored using a hash function
C) Records are in text (line sequential) format
D) The physical storage order of records is random

■ **Correct Answer: (A)**

Explanation: *A clustered index means data records are physically sorted in the same order as the index. There can be only one clustered index per file.*

Q72. MULTI-LEVEL INDEXING is introduced to:

A) Merge two separate files into one
B) Sort records physically
C) Replace hashing completely
D) Reduce disk accesses for large files by adding index levels above the data index

■ **Correct Answer: (D)**

Explanation: *For very large files, a single-level index may itself be large. Adding a higher-level index (index of indexes) reduces search time to O(log N) with fewer disk accesses.*

Q73. The CYLINDER INDEX in an ISAM file stores:

A) The highest key value on each cylinder
B) Overflow area pointers only
C) All data records
D) All hash values

■ **Correct Answer: (A)**

Explanation: *The ISAM cylinder index has one entry per cylinder, containing the largest key on that cylinder. The search algorithm uses this to identify the correct cylinder quickly.*

Q74. A specific DISADVANTAGE of ISAM compared to VSAM/B+ tree is:

A) Does not support sequential access
B) Overflow chains degrade performance as the file grows without reorganisation
C) Requires a hash function
D) Cannot be indexed

■ **Correct Answer: (B)**

Explanation: *ISAM is static. As records insert into full areas, overflow chains grow and require more disk accesses per record. Periodic file reorganisation is required to fix this.*

Q75. The ACCESS METHODS supported by indexed file organisation are:

A) Hashed access only
B) Sequential access only
C) Both sequential and random access
D) Random access only

■ **Correct Answer: (C)**

Explanation: *Indexed sequential files support sequential access (reading records in key order via the index) AND random access (using the index for direct retrieval).*

Q76. The INDEX FILE in indexed organisation contains:

A) Hash buckets
B) (Key, Disk_Address) pairs sorted by key
C) All data records
D) Only end-of-file markers

■ **Correct Answer: (B)**

Explanation: *The index stores (key, address) pairs. It is sorted by key so binary search can efficiently locate any key and retrieve the corresponding disk address.*

Q77. True/False: In indexed file organisation, the index MUST be updated when records are inserted or deleted.

A) True — the index must always reflect current data file state
B) True only for dense indexes
C) False
D) True only for B+ tree indexes

■ **Correct Answer: (A)**

Explanation: *Any change to the data file (insert, delete, key update) requires a corresponding index update to keep the index consistent with the actual data.*

Q78. In a B+ TREE index, DATA POINTERS are stored:

A) In external hash buckets
B) Only in leaf nodes
C) Equally in both internal and leaf nodes
D) Only in internal (non-leaf) nodes

■ **Correct Answer: (B)**

Explanation: *In a B+ tree, only leaf nodes hold data record pointers. Internal nodes hold only key values for routing search down the tree. Leaf nodes are also linked sequentially.*

Q79. The LINKED LEAF NODES in a B+ tree enable:

A) Hash collision resolution
B) Line-sequential newline-based access
C) Efficient sequential scans and range queries
D) Random hash-based access

■ **Correct Answer: (C)**

Explanation: *Linked leaf nodes allow sequential traversal of all records in key order — simply follow the leaf chain without climbing back up the tree.*

Q80. Indexed file organisation is BEST suited when the application needs:

A) Both frequent direct lookups AND periodic sorted reports
B) Only sequential batch processing with no individual lookups
C) Only equality searches with no sorted output required
D) Only text-based storage

■ **Correct Answer: (A)**

Explanation: *When an application needs random access (e.g., look up one customer by ID) AND sequential access (e.g., generate a sorted report), indexed organisation is the best single choice.*

Q81. In LINE SEQUENTIAL file organisation, records are separated by:

A) Fixed-length padding bytes
B) Newline (CR, LF, or CR+LF) characters
C) Index pointer values
D) Computed hash values

■ **Correct Answer: (B)**

Explanation: *Line sequential files are text files where each record occupies one line. The newline character acts as the record delimiter separating one record from the next.*

Q82. Line sequential files are typically stored as:

A) Human-readable TEXT files
B) Hashed binary files
C) Encrypted binary files
D) Random-access binary files

■ **Correct Answer: (A)**

Explanation: *Line sequential files are plain text — each record is a line of printable characters. Any standard text editor can open and edit them.*

Q83. The PRIMARY ACCESS METHOD for line sequential files is:

A) Sequential access
B) Hashed direct access
C) Random (direct) access
D) Indexed access

■ **Correct Answer: (A)**

Explanation: *Records in line sequential files have variable lengths (newlines), so their start addresses cannot be calculated arithmetically. The file must be read sequentially, line by line.*

Q84. An ADVANTAGE of line sequential file organisation is:

A) Simple, human-readable, easily created and edited by standard tools
B) Fast random access to any specific record
C) Best performance for real-time direct access
D) Automatic indexing of all records

■ **Correct Answer: (A)**

Explanation: *Line sequential files are plain text — easy to create, read, edit with any text editor, and transfer between different systems and platforms.*

Q85. A major DISADVANTAGE of line sequential files is:

A) They cannot be opened in any text editor
B) They cannot store multiple fields per record
C) Direct access to a specific record is inefficient — the entire file may need to be scanned
D) They require a hash function for all access

■ **Correct Answer: (C)**

Explanation: *Since records are variable-length, the start address of record N cannot be calculated. To reach record N, records 1 through N-1 must be read first.*

Q86. CSV (Comma-Separated Values) files are a classic example of:

A) Line sequential text files
B) Random-access binary files
C) Indexed sequential files
D) Hashed binary files

■ **Correct Answer: (A)**

Explanation: *A CSV file stores one record per line, with fields separated by commas. This is the quintessential line sequential format — text-based, one record per line.*

Q87. UPDATING a record IN THE MIDDLE of a line sequential file typically requires:

A) Reading all records, modifying the target, rewriting the entire (or remainder of the) file
B) Rebuilding the index structure
C) Applying a hash function to find the new address
D) Direct in-place overwrite using a file pointer

■ **Correct Answer: (A)**

Explanation: *Variable-length text records cannot be modified in-place (a longer replacement would overwrite the next record). The file must be rewritten with the updated content.*

Q88. On WINDOWS, line sequential files use which line ending?

A) CR only (\r)
B) CR+LF (\r\n)
C) Tab character (\t)
D) LF only (\n)

■ **Correct Answer: (B)**

Explanation: *Windows uses Carriage Return + Line Feed (\r\n) as the line ending. Unix/Linux uses LF (\n) only. This difference affects portability of line sequential files.*

Q89. Line sequential files are MOST suitable for:

A) Airline seat reservation requiring instant individual record access
B) System log files, configuration files, and text data
C) ATM account balance lookup
D) Bank transaction processing with random access

■ **Correct Answer: (B)**

Explanation: *Log files and configuration files are ideal uses: they are written sequentially (appended), read top-to-bottom, human-readable for administrators, and easily transferred.*

Q90. Which is NOT a feature of line sequential file organisation?

A) Hash-based direct record access
B) Text-based format
C) Newline character as record delimiter
D) One record per line

■ **Correct Answer: (A)**

Explanation: *Line sequential files do NOT support hash-based direct access. Their defining features are text format, one record per line, and newline delimiters.*

Q91. APPENDING a new record to a line sequential file:

A) Requires computing a hash function
B) Requires rewriting the entire file from scratch
C) Requires rebuilding the index
D) Simply adds a new line at the end — efficient and fast

■ **Correct Answer: (D)**

Explanation: *Appending is very efficient for line sequential files: open in append mode, write the new line, close. No existing content is disturbed.*

Q92. Which storage medium is MOST analogous to line sequential file access?

A) Hard disk with random access
B) Magnetic tape (serial access)
C) SSD with direct access
D) RAM

■ **Correct Answer: (B)**

Explanation: *Magnetic tape is inherently serial — you must spool through from the beginning to reach any position. Line sequential files have the same serial-access characteristic.*

Q93. Line sequential and pure sequential file organisation SHARE which property?

A) Both use newline characters as delimiters
B) Both use hash functions
C) Both require an index for access
D) Both support only sequential (serial) access

■ **Correct Answer: (D)**

Explanation: *Both line sequential and sequential organisation support sequential access. Line sequential adds the text-format and newline-delimiter specifics to this common property.*

Q94. Storing INTEGERS as text in a line sequential file compared to binary:

A) Always uses exactly the same space
B) Uses a hash table internally
C) Always uses less storage space
D) May use MORE space (e.g., '12345' needs 5 bytes vs 4-byte binary integer)

■ **Correct Answer: (D)**

Explanation: *Text representation of numbers uses one byte per digit. A 5-digit number uses 5 bytes as text but only 4 bytes as a binary integer. Large numbers cost more space in text.*

Q95. True/False: Line sequential files are suitable for REAL-TIME TRANSACTION PROCESSING needing instant individual record access.

A) True only for files with fewer than 100 records
B) True if the file is sorted
C) True
D) False — sequential scan makes large-file individual record retrieval slow

■ **Correct Answer: (D)**

Explanation: *Line sequential files require scanning from the beginning to find any record. For real-time systems needing instant access, this is unacceptably slow for large files.*

Q96. Which COMPARISON between line sequential and indexed organisation is CORRECT?

A) Indexed supports both sequential and random access; line sequential supports sequential access only
B) Line sequential always uses binary file format
C) Line sequential supports faster random access than indexed
D) Both support direct hash-based access

■ **Correct Answer: (A)**

Explanation: *Indexed files support both access modes. Line sequential supports only sequential access. This fundamental difference determines when each is appropriate.*

Q97. Fields WITHIN ONE LINE (record) of a line sequential file are typically separated by:

A) A field delimiter such as comma, tab, or pipe character
B) Another newline character
C) A hash symbol
D) A null byte (0x00)

■ **Correct Answer: (A)**

Explanation: *While newlines separate records, fields within a record are separated by a field delimiter: comma (CSV), tab (TSV), or pipe (\|). Example: John,25,Delhi*

Q98. True/False: Line sequential files can store records with MULTIPLE FIELDS.

A) True — multiple fields per line/record are separated by a field delimiter
B) True only for fixed-length field files
C) True only for binary line sequential files
D) False — only one field per line is possible

■ **Correct Answer: (A)**

Explanation: *Yes. Multiple fields per line are standard (e.g., CSV: name,age,city on one line = three fields in one record).*

Q99. In HASHED file organisation, a record's storage address is determined by:

A) The record's line number in the file
B) Sequential scanning from the file start
C) An index table lookup
D) Applying a mathematical hash function to the record's key

■ **Correct Answer: (D)**

Explanation: *A hash function h(key) computes the storage address (bucket number) directly from the key. No index search or sequential scan is required.*

Q100. A 'BUCKET' in hashed file organisation is:

A) An index entry
B) A single record
C) A line in a text file
D) A storage unit (one or more blocks) that can hold one or more records

■ **Correct Answer: (D)**

Explanation: *A bucket is a storage unit whose size is typically one or more disk blocks. Records whose keys hash to the same address are stored in the same bucket.*

Q101. When two DIFFERENT KEYS produce the same hash address, this is called:

A) Block overflow
B) Duplication
C) Collision
D) Fragmentation

■ **Correct Answer: (C)**

Explanation: *A collision (synonym situation) occurs when h(key1) = h(key2) for two different keys. Collision resolution techniques (chaining, probing) are required to handle this.*

Q102. The DIVISION-REMAINDER hash method calculates the address as:

A) sum of folded key parts
B) key mod number_of_buckets
C) key² (middle digits extracted)
D) key × number_of_buckets

■ **Correct Answer: (B)**

Explanation: *Division-remainder: h(k) = k mod m, where m = number of buckets. Example: h(250) = 250 mod 100 = 50.*

Q103. For h(k) = k mod 7 and k = 49, the hash address is:

A) 49
B) 1
C) 7
D) 0

■ **Correct Answer: (D)**

Explanation: *49 mod 7 = 7 × 7 = 49, remainder = 0. So h(49) = 0.*

Q104. The FOLDING hash method involves:

A) Dividing the key by the number of buckets
B) Squaring the key and extracting middle digits
C) Sorting the digits of the key
D) Splitting the key into equal parts and adding or XOR-ing them

■ **Correct Answer: (D)**

Explanation: *Folding splits a long key into equal-sized parts (e.g., a 9-digit key into three 3-digit parts) and adds or XORs them to get a hash address.*

Q105. The MID-SQUARE hash method involves:

A) Dividing by a prime number
B) Using an index to refine the address
C) Folding key parts together
D) Squaring the key and selecting middle digits as the address

■ **Correct Answer: (D)**

Explanation: *Mid-square: square the key, then extract the middle digits of the result as the hash address. E.g., key=12, 12²=144, middle digit=4, so address=4.*

Q106. CHAINING as a collision resolution technique stores overflow records:

A) Using a secondary hash function
B) In the next sequential available slot
C) In a linked list attached to the home bucket
D) In a separate sorted overflow file

■ **Correct Answer: (C)**

Explanation: *Chaining (separate chaining) maintains a linked list of all records that hash to the same bucket. The bucket contains a pointer to its overflow chain.*

Q107. LINEAR PROBING resolves collisions by:

A) Trying the next slot sequentially (slot+1, slot+2, ...)
B) Creating a new overflow file
C) Applying a second independent hash function
D) Using a linked list

■ **Correct Answer: (A)**

Explanation: *Linear probing: if h(k) is occupied, try h(k)+1, h(k)+2, ... (wrapping around) until an empty slot is found. Simple but may cause primary clustering.*

Q108. A PERFECT HASH FUNCTION:

A) Maps every distinct key to a unique address with zero collisions
B) Always requires an overflow area
C) Always uses the division method
D) Causes maximum collisions

■ **Correct Answer: (A)**

Explanation: *A perfect hash function produces no collisions — every key gets a unique address. Achievable only when the complete set of keys is known at design time.*

Q109. The LOAD FACTOR of a hash file is:

A) Number of collisions recorded so far
B) The number of hash functions used
C) The size of each bucket in bytes
D) Ratio of records stored to total available slots

■ **Correct Answer: (D)**

Explanation: *Load factor $\alpha = n/m$ (n = records stored, m = total slots). High $\alpha \rightarrow$ more collisions $\rightarrow$ longer search times.*

Q110. The PRIMARY ADVANTAGE of hashed file organisation is:

A) Automatically sorts records on insertion
B) Approximately O(1) direct access to individual records
C) No key field is required
D) Efficient sequential access in sorted key order

■ **Correct Answer: (B)**

Explanation: *Hashing's main advantage is O(1) average-case retrieval — for a known key, one hash computation gives the bucket address, ideally one disk access retrieves the record.*

Q111. The PRIMARY DISADVANTAGE of hashed file organisation is:

A) Cannot handle large files
B) Poor support for sequential processing and range queries
C) Slow direct access to individual records
D) Requires an external index for all access

■ **Correct Answer: (B)**

Explanation: *Hash functions do not preserve key order, so records are not stored in any meaningful sequence. Sequential reports and range queries are very inefficient.*

Q112. Hashed file organisation is also called:

A) Line sequential organisation
B) Indexed sequential organisation
C) VSAM organisation
D) Direct file organisation

■ **Correct Answer: (D)**

Explanation: *Direct file organisation = hashed organisation. The hash function provides a direct computation of the storage address, bypassing scanning and indexing.*

Q113. Two different keys mapping to the SAME hash address are called:

A) Synonyms
B) Duplicate records
C) Cluster keys
D) Redundant keys

■ **Correct Answer: (A)**

Explanation: *In hashing terminology, keys that produce the same hash address are called synonyms. The event is called a collision.*

Q114. DOUBLE HASHING resolves collisions by:

A) Linear probing with step size 1
B) Using a linked overflow list
C) Creating a separate overflow file
D) Applying a second independent hash function to determine the probe step increment

■ **Correct Answer: (D)**

Explanation: *Double hashing: if h1(k) is occupied, probe at h1(k) + i×h2(k) for i=1,2,3... The variable step from h2 reduces clustering better than linear probing.*

Q115. For a hashed file, the number of buckets should ideally be:

A) A power of 2 only
B) Equal to the number of records
C) A prime number
D) An even number

■ **Correct Answer: (C)**

Explanation: *A prime number of buckets ensures the modulo operation distributes keys more uniformly, reducing the probability of collision.*

Q116. True/False: Hashed file organisation stores records in SORTED KEY ORDER.

A) True when load factor is below 0.5
B) True
C) False — hash functions scatter records with no preserved key order
D) True only for perfect hash functions

■ **Correct Answer: (C)**

Explanation: *Hash functions map keys to pseudo-random bucket addresses. Records with adjacent key values may end up in completely different buckets — no sorted order is maintained.*

Q117. Hashed access is LEAST SUITABLE for:

A) Checking if a specific key exists
B) Single-record direct retrieval
C) Equality queries (find record with key = X)
D) Range queries (find all records with X ≤ key ≤ Y)

■ **Correct Answer: (D)**

Explanation: *Hash functions destroy key order, making range queries require scanning all buckets — essentially O(N). This is hashing's main weakness.*

Q118. Which collision resolution technique stores overflow records in a LINKED LIST?

A) Chaining (separate chaining)
B) Linear probing
C) Quadratic probing
D) Double hashing

■ **Correct Answer: (A)**

Explanation: *Separate chaining attaches a linked list to each bucket. All records that hash to the same bucket are stored in that bucket's chain. Unlimited overflow capacity.*

Q119. REHASHING involves:

A) Adding an index to the hash file
B) Creating a new larger hash table and reinserting all existing records via new hash function
C) Merging two hash files
D) Sorting the existing hash table

■ **Correct Answer: (B)**

Explanation: *Rehashing is triggered when load factor becomes too high. A new larger table is created and all existing keys are re-hashed into it, redistributing records.*

Q120. Match: File Organisation → Key Characteristic 1-Indexed 2-Line Sequential 3-Hashed A-Hash function computes address B-Index table maps keys to addresses C-Newline delimiters separate records

A) 1-A, 2-B, 3-C

B) 1-B, 2-A, 3-C

C) 1-B, 2-C, 3-A

D) 1-C, 2-A, 3-B

■ **Correct Answer: (C)**

Explanation: *Indexed uses an index table (B). Line Sequential uses newline delimiters (C). Hashed uses a hash function to compute the address (A).*

SECTION 3: FILE ACCESSING TECHNIQUES

Q121. In SEQUENTIAL ACCESS, records are read:

A) Via an index lookup table
B) One after another in the physical stored order
C) In any order by specifying a key value
D) By applying a hash function to the key

■ **Correct Answer: (B)**

Explanation: *Sequential access reads records serially from first to last in the order they are physically stored. No skipping is possible without reading all intervening records.*

Q122. Sequential access is also known as:

A) Direct access
B) Indexed access
C) Serial access
D) Random access

■ **Correct Answer: (C)**

Explanation: *Sequential = serial access. Records are read one after another in series, like playing a cassette tape from start to end.*

Q123. Sequential access is MOST EFFICIENT when:

A) A large proportion of all records must be processed in one run
B) One specific record needs fast retrieval
C) Records are accessed by a hash key
D) An index provides shortcuts to records

■ **Correct Answer: (A)**

Explanation: *When the activity ratio is high (many or all records needed), sequential access processes the file in one continuous pass — minimum disk head movement, maximum throughput.*

Q124. The AVERAGE number of records read to find a specific record by sequential access in a file of N records is:

A) log■N
B) N/2
C) N
D) 1

■ **Correct Answer: (B)**

Explanation: *On average, the target is in the middle of the file, requiring N/2 reads. Worst case (last record or not found) = N reads.*

Q125. Which storage device INHERENTLY supports sequential access only?

A) Magnetic tape
B) RAM
C) Solid State Drive (SSD)
D) Hard disk drive (HDD)

■ **Correct Answer: (A)**

Explanation: *Magnetic tape is a serial-access medium. To reach any point, the tape must spool forward from the current position. Disks and SSDs support both sequential and random access.*

Q126. Which application BEST suits sequential access?

A) Airline seat reservation system
B) Monthly payroll processing for ALL employees
C) ATM account balance inquiry
D) Real-time stock price lookup

■ **Correct Answer: (B)**

Explanation: *Payroll reads every employee record monthly — high activity ratio. Sequential access processes the entire file efficiently in one pass.*

Q127. The time complexity of sequential search in a file of N records is:

A) $O(N^2)$
B) O(1)
C) O(log N)
D) O(N)

■ **Correct Answer: (D)**

Explanation: *Sequential search examines records one by one from the start. Worst case: all N records read. Average case: N/2. Both are O(N).*

Q128. True/False: Sequential access can efficiently retrieve the LAST RECORD without reading all preceding records.

A) False — sequential access must read every preceding record to reach the last one
B) True only on disk (not tape)
C) True
D) True if the file is sorted

■ **Correct Answer: (A)**

Explanation: *Sequential access has no shortcut. Every record from the first must be read to reach the last. This is its fundamental limitation for individual record retrieval.*

Q129. Batch processing is associated with which access method?

A) Line sequential with index
B) Sequential access
C) Random indexed access
D) Hashed direct access

■ **Correct Answer: (B)**

Explanation: *Batch processing reads and processes large volumes of records in sequence. Sequential access is the natural and most efficient method for batch jobs.*

Q130. In INDEXED ACCESS, how is a record retrieved?

A) The file pointer advances by a fixed record_length
B) The index is searched for the key to get the disk address, then the record is fetched
C) The hash function computes the address directly
D) All records are scanned sequentially until found

■ **Correct Answer: (B)**

Explanation: *Indexed access: (1) search the sorted index for the key (binary search), (2) use the retrieved disk address to directly fetch the data record.*

Q131. Indexed access is FASTER than sequential access for single-record retrieval because:

A) Records are stored in random order
B) The index eliminates the need to read every record before the target
C) No key field is needed
D) It uses a hash function for computation

■ **Correct Answer: (B)**

Explanation: *The index provides a direct shortcut to the record's address. Instead of N/2 average reads, only index access + one data block read are needed.*

Q132. Binary search on an index is possible because:

A) Index entries are stored in a hash table
B) Index stores line numbers for each record
C) Index uses doubly-linked lists
D) Index entries are SORTED by key value

■ **Correct Answer: (D)**

Explanation: *A sorted index allows binary search to halve the search space with each comparison, finding the key in O(log N) comparisons instead of O(N).*

Q133. Indexed access supports which query type that HASHED access does NOT support efficiently?

A) Direct single-record retrieval by exact key
B) Equality query (find record with key = X)
C) Range query (find all records with X ≤ key ≤ Y)
D) Single-pass full-file read

■ **Correct Answer: (C)**

Explanation: *Because the index is ordered by key, range queries can be answered by finding the range start in the index and traversing sequentially to the range end. Hashing destroys key order.*

Q134. In a MULTI-LEVEL INDEX, the search time is approximately:

A) O(1)
B) $O(N^2)$
C) O(N)
D) O(log N)

■ **Correct Answer: (D)**

Explanation: *Each level of the multi-level index reduces the search space by a factor, giving O(log N) overall — similar to binary search but across disk accesses.*

Q135. A DISADVANTAGE of indexed access compared to sequential access is:

A) Requires extra storage for the index; index must be maintained on every update
B) Does not support sorted output
C) Cannot retrieve individual records
D) Always slower than sequential for individual retrieval

■ **Correct Answer: (A)**

Explanation: *Indexes consume additional disk space. Every write operation (insert/delete/key update) must also update the index, adding overhead to modifications.*

Q136. True/False: Indexed access always requires reading every record before finding the target.

A) True only for sparse indexes
B) True for files with more than 1,000 records
C) True
D) False — the index provides the disk address directly, skipping all other records

■ **Correct Answer: (D)**

Explanation: *Indexed access uses the index to jump directly to the record's location, reading only the index and the target data block.*

Q137. Indexed access enables RANGE QUERIES because:

A) Line numbers provide range information
B) Hash functions preserve key order
C) The sorted index can be traversed from range start to range end sequentially
D) Records are physically stored in a hash table

■ **Correct Answer: (C)**

Explanation: *A sorted index supports range queries: find the start key in the index (binary search), then follow index entries sequentially to the end of the range.*

Q138. In HASHED ACCESS, a record's storage address is determined by:

A) Following a linked list
B) Applying a hash function to the key
C) Scanning the sorted index
D) Reading records one by one

■ **Correct Answer: (B)**

Explanation: *Hashed access computes the address arithmetically using h(key) = address. One computation gives the location — no index search or sequential scan needed.*

Q139. Hashed access is also called:

A) Serial access
B) Direct access
C) Indexed access
D) Line sequential access

■ **Correct Answer: (B)**

Explanation: *Hashed access = direct access. The hash function directly computes the storage address, allowing one-step retrieval in the ideal (no-collision) case.*

Q140. The AVERAGE number of disk accesses for hashed access with NO COLLISION is:

A) 1
B) log■N
C) N/2
D) N

■ **Correct Answer: (A)**

Explanation: *With no collision, h(key) immediately gives the bucket address. One disk access reads the bucket and retrieves the record — O(1) performance.*

Q141. Hashed access is LEAST SUITABLE for:

A) Equality queries (key = X)
B) Single-record direct retrieval
C) Range queries (X ≤ key ≤ Y)
D) Checking if a specific key exists

■ **Correct Answer: (C)**

Explanation: *Hash functions do not preserve key order. A range query requires visiting all buckets — O(N) — essentially no better than sequential search.*

Q142. Which access method provides the FASTEST average retrieval for a single known key?

A) Hashed O(1)
B) Indexed O(log N)
C) Line sequential O(N)
D) Sequential O(N)

■ **Correct Answer: (A)**

Explanation: *Hashed access achieves O(1) average-case retrieval — one hash computation gives the address, ideally one disk access retrieves the record.*

Q143. True/False: Hashed access always results in EXACTLY ONE disk access per retrieval.

A) True
B) True only for small files
C) True only for perfect hash functions
D) False — collisions require additional accesses for overflow chains or probing

■ **Correct Answer: (D)**

Explanation: *Collisions mean the record is not in its home bucket. Overflow chains or probe sequences require additional disk accesses beyond the first.*

Q144. An ATM system retrieving an account by account number BEST uses:

A) Hashed direct access
B) Line sequential
C) Read all records then filter
D) Sequential scan

■ **Correct Answer: (A)**

Explanation: *ATM systems need instant individual record retrieval. Hashed direct access provides O(1) lookup by account number — ideal for this real-time requirement.*

Q145. Correct ordering of access methods for SINGLE-RECORD RETRIEVAL speed (fastest first):

A) Sequential → Indexed → Hashed
B) Hashed (O(1)) → Indexed (O(log N)) → Sequential (O(N))
C) All three are equally fast
D) Indexed → Hashed → Sequential

■ **Correct Answer: (B)**

Explanation: *For individual record retrieval: Hashed O(1) is fastest, Indexed O(log N) is middle, Sequential O(N) is slowest.*

Q146. For BATCH PROCESSING ALL EMPLOYEES weekly, the MOST EFFICIENT access is:

A) Indexed random access
B) Hashed direct access
C) Sequential access
D) Line sequential with no index

■ **Correct Answer: (C)**

Explanation: *Processing all records (100% activity ratio) in sequence is most efficient with sequential access — one continuous pass through the file.*

Q147. For a LIBRARY LOOKUP by ISBN (exact match), the MOST EFFICIENT access is:

A) Line sequential
B) Sequential scan
C) Hashed direct access
D) No special access needed

■ **Correct Answer: (C)**

Explanation: *ISBN lookup is an equality search — ideal for hashing. One hash computation gives the book's location directly.*

Q148. For a UNIVERSITY needing both sorted student lists AND individual lookups by roll number, the BEST is:

A) Line sequential only
B) Pure sequential
C) Pure hashed
D) Indexed sequential (ISAM/VSAM)

■ **Correct Answer: (D)**

Explanation: *Indexed sequential supports both: the index enables direct roll-number lookup; sorted physical storage enables efficient sequential reports.*

Q149. Assertion (A): Hashed access is the BEST method for sorted report generation. Reason (R): Hash functions produce record addresses in sorted key order.

A) A is FALSE and R is FALSE — hash functions do NOT preserve order
B) A is false; R is true
C) A is true; R is false
D) Both A and R are true

■ **Correct Answer: (A)**

Explanation: *Both the assertion and the reason are false. Hash functions scatter records with no sorted order. Hashing is very poor for generating sorted reports.*

Q150. Assertion (A): Indexed access supports range queries. Reason (R): The sorted index allows efficient traversal of a contiguous key range.

A) Both are false
B) Both A and R are true, and R correctly explains A
C) A is true; R is false
D) A is false; R is true

■ **Correct Answer: (B)**

Explanation: *Both are correct. The sorted index supports range queries (A) precisely because it can be traversed from range-start to range-end sequentially (R explains A).*

Q151. Match: Access Technique → Best Use Case 1-Sequential 2-Indexed 3-Hashed A-ATM account withdrawal B-Monthly payroll processing C-Student report by name range

A) 1-B, 2-C, 3-A
B) 1-A, 2-B, 3-C
C) 1-B, 2-A, 3-C
D) 1-C, 2-A, 3-B

■ **Correct Answer: (A)**

Explanation: *Sequential is best for full-file batch (payroll → B). Indexed is best for range queries (student name range → C). Hashed is best for single equality lookup (ATM → A).*

Q152. Which access method requires NO EXTRA STORAGE OVERHEAD (no index, no hash table)?

A) ISAM access
B) Hashed access
C) Sequential access
D) Indexed access

■ **Correct Answer: (C)**

Explanation: *Pure sequential access requires only the data file itself — no index structure and no hash table. Lowest possible storage overhead.*

Q153. True/False: Sequential access is the only access method with NO STORAGE OVERHEAD.

A) False
B) True — sequential access needs only the data file itself
C) True only for unsorted files
D) True only for text files

■ **Correct Answer: (B)**

Explanation: *Sequential access requires no additional structures (no index, no hash table). It operates directly on the data file with zero additional overhead.*

Q154. Which access method is BEST when 90% of records must be processed in every run?

A) Hashed
B) Line sequential with index
C) Indexed
D) Sequential

■ **Correct Answer: (D)**

Explanation: *90% activity ratio means nearly all records are needed. Sequential access reads them all efficiently in one continuous pass, whereas random access methods would cause excessive seeks.*

Q155. True/False: Indexed access supports BOTH reading records in sorted key order AND direct access to individual records.

A) True only for B+ trees
B) True only for ISAM
C) False
D) True — the sorted index enables both modes simultaneously

■ **Correct Answer: (D)**

Explanation: *Because the index is sorted by key, traversing it sequentially gives records in key order (sequential); using it for lookup gives direct access to any record.*

Q156. A BANK needing monthly sorted account statements AND instant account lookup SHOULD use:

A) Pure hashed only
B) Pure sequential only
C) Indexed sequential (ISAM/VSAM)
D) Line sequential only

■ **Correct Answer: (C)**

Explanation: *ISAM/VSAM provides: indexed access for instant account lookup AND sequential access for monthly sorted statements. No other single organisation serves both as efficiently.*

Q157. True/False: Hashed access can efficiently process records in ASCENDING KEY ORDER.

A) True only for integer keys
B) True
C) False — hashing destroys key order; ascending-order processing requires scanning all buckets
D) True if the hash function is linear

■ **Correct Answer: (C)**

Explanation: *Hash functions map keys to pseudo-random addresses. Processing records in key order using a hash file requires visiting all buckets in sequence — essentially O(N).*

Q158. In hashed access, when a COLLISION OCCURS, which of the following increases?

A) The hash function changes
B) The number of index levels
C) The number of disk accesses needed to retrieve the record
D) The load factor decreases

■ **Correct Answer: (C)**

Explanation: *Collision means the record is not in its home bucket. The collision resolution mechanism (chain traversal, probing) requires additional disk accesses beyond the first.*

Q159. True/False: Sequential access is still widely used in modern computing.

A) False — sequential access is completely obsolete
B) True — log files, batch processing, data backups all use sequential access
C) True only for tape-based systems
D) True only for mainframe systems

■ **Correct Answer: (B)**

Explanation: *Sequential access remains essential: log files are written/read sequentially; batch jobs process entire files; data backups stream data sequentially. It is NOT obsolete.*

Q160. Scenario: A government database stores 10 million voter records. An election officer needs to print all records sorted alphabetically. Which access method is BEST?

A) Hashed direct access — one by one
B) Sequential access on a file sorted by name
C) Random indexed access — one by one
D) Line sequential with no index

■ **Correct Answer: (B)**

Explanation: *Printing all 10 million records (100% activity ratio) in alphabetical order is best done by sequential access on a name-sorted file — one efficient continuous pass.*

SECTION 4: FILE HANDLING FUNCTIONS — SORTING, MERGING, INDEXING, UPDATING

Q161. File SORTING is the process of:

A) Combining two sorted files into one
B) Computing hash addresses for records
C) Adding new records to a file
D) Arranging records of a file in a specified order based on a key field

■ **Correct Answer: (D)**

Explanation: *Sorting rearranges records into ascending or descending order on a sort key, enabling binary search, ordered reports, and efficient merging.*

Q162. An INTERNAL SORT operates:

A) Using a hash function for placement
B) On files too large for RAM, using disk storage
C) Using an external merge technique
D) Entirely within main memory (RAM)

■ **Correct Answer: (D)**

Explanation: *Internal sorting algorithms (quicksort, heapsort, merge sort) operate entirely in RAM. They require the complete data set to fit in available memory.*

Q163. An EXTERNAL SORT is needed when:

A) The file fits entirely in RAM
B) The file is too large to fit in main memory
C) The file uses hashing
D) The file is already perfectly sorted

■ **Correct Answer: (B)**

Explanation: *When a file exceeds available RAM, it cannot be sorted in memory alone. External sorting uses disk-based techniques (create sorted runs, then merge) to handle files larger than RAM.*

Q164. The SORT KEY is:

A) An index field only
B) The field (attribute) on which records are to be sorted
C) The physical record number
D) The hash field of the file

■ **Correct Answer: (B)**

Explanation: *The sort key determines the ordering criterion. Example: sort employees by Employee_ID (ascending) or by Salary (descending).*

Q165. A MAJOR KEY in multi-field sorting is:

A) A field used only for indexing, not sorting
B) The hash key of the record
C) The secondary sort field used for tie-breaking
D) The primary sort field with highest priority

■ **Correct Answer: (D)**

Explanation: *When sorting on multiple fields, the major key is compared first. The minor key resolves ties when major key values are equal.*

Q166. A MINOR KEY in multi-field sorting is:

A) Used to break ties when major key values are equal across records
B) The hash function input
C) The file header identifier field
D) The primary sort field

■ **Correct Answer: (A)**

Explanation: *Example: sort students by City (major key), then by Name (minor key) within each city. Minor key only matters when major keys are equal.*

Q167. BUBBLE SORT works by:

A) Inserting each element at its correct position
B) Finding the minimum and placing it first
C) Repeatedly swapping adjacent elements that are in the wrong order
D) Dividing data into halves and sorting each recursively

■ **Correct Answer: (C)**

Explanation: *Bubble sort compares adjacent records and swaps them if out of order, causing the largest unsorted element to 'bubble up' to its correct position each pass.*

Q168. SELECTION SORT works by:

A) Inserting each element at its correct position
B) Swapping adjacent elements
C) Finding the minimum element in the unsorted portion and placing it in the correct position
D) Applying a hash function

■ **Correct Answer: (C)**

Explanation: *Selection sort scans the unsorted portion, selects the minimum, swaps it to the correct position. Simple but always makes $O(N^2)$ comparisons.*

Q169. INSERTION SORT works by:

A) Finding and swapping adjacent pairs
B) Taking one element at a time and inserting it into its correct position among already-sorted elements
C) Selecting the minimum each pass
D) Dividing and merging

■ **Correct Answer: (B)**

Explanation: *Insertion sort builds the sorted sequence one element at a time. Each new element is inserted into its correct position in the already-sorted portion. Efficient for small or nearly-sorted data.*

Q170. MERGE SORT is preferred for EXTERNAL FILE SORTING because:

A) It uses hash functions for placement
B) It requires no extra storage space
C) It sorts in O(N) time
D) Its merge phase reads files sequentially — ideal for disk and tape

■ **Correct Answer: (D)**

Explanation: *Merge sort's merge step reads two sorted sequences sequentially — perfectly suited for disk and tape I/O. It also guarantees O(N log N) performance.*

Q171. In external merge sort, a 'RUN' is:

A) A hash bucket
B) An index entry in the index file
C) A sorted sequence of records (sorted sub-file)
D) A single data record

■ **Correct Answer: (C)**

Explanation: *A run is a contiguous sorted sequence created during the run-generation phase. Multiple runs are then merged to produce the final sorted file.*

Q172. In a TWO-WAY MERGE, runs are combined:

A) 4 at a time
B) 1 at a time (no merge)
C) All at once
D) 2 at a time

■ **Correct Answer: (D)**

Explanation: *Two-way (binary) merge takes 2 sorted runs and produces 1 larger sorted run each pass. After ■log■(R)■ passes, one final sorted file remains.*

Q173. The TIME COMPLEXITY of merge sort is:

A) O(N log N)
B) O(1)
C) O(N^2)
D) O(N)

■ **Correct Answer: (A)**

Explanation: *Merge sort divides data into log N levels and performs O(N) work at each level → O(N log N) total. This is the optimal bound for comparison-based sorting.*

Q174. A STABLE SORT preserves:

A) The physical layout of the file
B) The index structure
C) The original relative order of records that have equal key values
D) The hash order of records

■ **Correct Answer: (C)**

Explanation: *Stability ensures that records with equal keys remain in their original relative order after sorting. Merge sort is stable; basic quicksort is not.*

Q175. Which sorting algorithm is NOT STABLE by default?

A) Merge sort
B) Insertion sort
C) Quick sort (basic pivot)
D) Bubble sort

■ **Correct Answer: (C)**

Explanation: *Basic quicksort is not stable — partitioning can change the relative order of equal-key records. Merge sort, insertion sort, and bubble sort are stable.*

Q176. The number of PASSES in external merge sort affects:

A) The hash collision rate
B) Total disk I/O time (each pass reads and writes the entire file)
C) The index size
D) The record length only

■ **Correct Answer: (B)**

Explanation: *Each pass reads and writes every record once. Minimising passes (by using higher-order merges) directly reduces total I/O time, which dominates external sort performance.*

Q177. In the RUN GENERATION phase of external sort:

A) An index is built on the data
B) Records are read into RAM, sorted in memory, written as sorted runs to disk
C) Sorted runs are merged into larger files
D) The file is reorganised using hashing

■ **Correct Answer: (B)**

Explanation: *Run generation (sorting phase): load as many records as fit in RAM, sort them (quicksort/heapsort), write the sorted chunk (run) to disk. Repeat until all data is in runs.*

Q178. In the MERGE PHASE of external sort:

A) Records are re-indexed
B) New initial sorted runs are created
C) Sorted runs from disk are progressively merged into larger sorted files
D) Hash addresses are recomputed

■ **Correct Answer: (C)**

Explanation: *The merge phase takes the sorted runs produced in the run-generation phase and repeatedly merges them (2-way or N-way) until one final completely sorted file remains.*

Q179. Sorting a file BEFORE MERGING ensures:

A) No hash collisions occur
B) The merged output file is also correctly sorted
C) Hashing is no longer needed
D) The file uses less storage space

■ **Correct Answer: (B)**

Explanation: *Merging only produces a correctly sorted output when both input files are sorted on the same key. Sorting is the mandatory prerequisite for correct merging.*

Q180. True/False: Sorting physically rearranges records on disk in the new order.

A) False — sorting only changes an index, not the physical file
B) True only for indexed files
C) True only for files under 1 MB
D) True — a physical sort actually rewrites records in their new order on disk

■ **Correct Answer: (D)**

Explanation: *A physical sort rewrites the file with records in the new order. An index-based (logical) sort only reorders the index without moving data records.*

Q181. A 'TAG SORT' sorts only:

A) Line numbers in the text file
B) Keys and address pointers (tags) rather than the actual large records
C) Hash values only
D) The data records physically on disk

■ **Correct Answer: (B)**

Explanation: *Tag sort avoids moving large records. Small (key, address) pairs are sorted. The data file is then read in sorted order using the sorted tags. Far less I/O than physical sort.*

Q182. Which sorting algorithm performs BEST on NEARLY-SORTED data?

A) Basic quicksort
B) Insertion sort
C) Selection sort
D) Heap sort

■ **Correct Answer: (B)**

Explanation: *Insertion sort requires almost no shifting when data is nearly sorted, approaching O(N) performance on nearly-sorted input — far better than its $O(N^2)$ worst case.*

Q183. File MERGING is the process of:

A) Combining two or more SORTED files into a single SORTED output file
B) Splitting one file into multiple smaller files
C) Sorting a single file
D) Computing hash addresses for records from multiple files

■ **Correct Answer: (A)**

Explanation: *Merging takes multiple sorted input files and produces one sorted output by comparing records key-by-key and writing the smaller key first to the output.*

Q184. For successful merging, the input files MUST be:

A) In text (line sequential) format
B) In hashed binary format
C) Of exactly equal size
D) Sorted on the SAME key field

■ **Correct Answer: (D)**

Explanation: *The merge algorithm compares records from all input files by key. Both files must be sorted on the same key to produce a correctly sorted output.*

Q185. The OUTPUT of a merge operation is:

A) A single sorted file containing ALL records from all input files
B) Two separate sorted files
C) A hash table
D) An index file

■ **Correct Answer: (A)**

Explanation: *The merge output is one larger sorted file. It contains every record from all input files, arranged in correct key order.*

Q186. TWO-WAY MERGE combines:

A) Three files at once
B) Unsorted files directly
C) Four files simultaneously
D) Two sorted files into one sorted file

■ **Correct Answer: (D)**

Explanation: *Two-way (binary) merge takes exactly two sorted input files and produces one sorted output. It is the simplest form of merging.*

Q187. N-WAY MERGE combines:

A) Exactly two files
B) Files in random unsorted order
C) N sorted files simultaneously into one sorted output file
D) Unsorted files using hashing

■ **Correct Answer: (C)**

Explanation: *N-way merge processes N sorted input files simultaneously, using a priority queue to always select the record with the smallest key from any of the N inputs.*

Q188. The data structure used to implement efficient N-WAY MERGING is:

A) Queue
B) Priority queue (min-heap)
C) Stack
D) Hash table

■ **Correct Answer: (B)**

Explanation: *A min-heap of size N holds the current smallest record from each of N input files. Extracting the minimum and inserting the next record runs in O(log N).*

Q189. Merging is efficient for sequential files because:

A) It uses a hash function for speed
B) It requires random access to both files
C) Both files are read in a single left-to-right sequential pass
D) It requires an index to work

■ **Correct Answer: (C)**

Explanation: *Two sorted files can be merged in one sequential pass each — read one record from each, write the smaller, advance that file's pointer. Very efficient sequential I/O.*

Q190. True/False: Merging can be performed DIRECTLY on UNSORTED files.

A) True only for text files
B) True
C) False — both input files must be sorted on the same key before merging
D) True if files are small enough for RAM

■ **Correct Answer: (C)**

Explanation: *Merging relies on key-by-key sorted comparison. Unsorted input produces unsorted (incorrect) output. Sort both files first, then merge.*

Q191. Merging a file of 600 records with a file of 400 records produces:

A) 200 records
B) 400 records
C) 600 records
D) 1,000 records

■ **Correct Answer: (D)**

Explanation: *Merging combines all records from both files without deletion. 600 + 400 = 1,000 records in the output file.*

Q192. Assertion (A): File merging requires both input files to be sorted on the same key. Reason (R): During merging, records from both files are compared key-by-key to produce sorted output.

A) A is true; R is false
B) Both A and R are true, and R correctly explains A
C) A is false; R is true
D) Both are false

■ **Correct Answer: (B)**

Explanation: *The requirement for sorted input (A) is directly because the merge algorithm works by comparing keys sequentially (R). Both true; R explains A.*

Q193. In a merge, when BOTH files have a record with the SAME KEY:

A) A collision error is generated
B) Both records are written to the output in sorted order
C) One record is discarded as a duplicate
D) The merge operation stops

■ **Correct Answer: (B)**

Explanation: *Merging allows duplicate keys — both records are written to the output file. The sorted order is maintained for both.*

Q194. The merge phase of external sort uses merging to:

A) Hash records for direct access
B) Create the initial sorted runs
C) Combine sorted runs from the run-generation phase into the final sorted file
D) Build an index on the sorted file

■ **Correct Answer: (C)**

Explanation: *After run generation produces many small sorted runs on disk, the merge phase progressively combines them (2-way or k-way) until one final sorted file remains.*

Q195. A REAL-WORLD application of file merging is:

A) Hashing employee records for direct access
B) Updating a single account balance
C) Combining sorted sales files from three regional offices into one national sorted file
D) Indexing a product catalogue

■ **Correct Answer: (C)**

Explanation: *Merging is used to combine separate sorted files (from branches/departments) into one consolidated sorted file. This is a standard use case in batch file processing.*

Q196. The PRIMARY PURPOSE of INDEXING in a file is to:

A) Sort records physically on disk
B) Merge two separate files
C) Hash records for direct storage
D) Speed up record retrieval by providing a fast-access lookup structure

■ **Correct Answer: (D)**

Explanation: *An index is a separate sorted structure mapping keys to record addresses. It dramatically speeds retrieval by avoiding full sequential scans.*

Q197. A PRIMARY INDEX is based on:

A) A computed hash function
B) The file line number
C) The primary key of the data records
D) A non-key attribute of the records

■ **Correct Answer: (C)**

Explanation: *The primary index uses the primary key field — the unique identifier for each record. It allows direct retrieval of any record by its primary key.*

Q198. A SECONDARY INDEX is based on:

A) The total record count
B) A non-primary (secondary) key field
C) A hash value
D) The primary key

■ **Correct Answer: (B)**

Explanation: *A secondary index enables fast lookup by non-primary attributes (e.g., find all employees by department). Unlike the primary index, secondary keys may not be unique.*

Q199. A DENSE INDEX has:

A) One entry per hash bucket
B) One entry per data block
C) One entry per field in the record
D) One entry per data record

■ **Correct Answer: (D)**

Explanation: *A dense index has exactly one entry per record. Every record has a corresponding index entry, allowing any record to be found directly from the index.*

Q200. A SPARSE INDEX has:

A) Entries only for deleted records
B) One entry per hash collision
C) One entry per data block (block-level index)
D) One entry per record

■ **Correct Answer: (C)**

Explanation: *A sparse index has one entry per block. Smaller than a dense index but after locating the block via the index, a brief sequential scan within the block may be needed.*

Q201. Rebuilding all indexes after major file changes is called:

A) Remerging
B) Resorting
C) Rehashing
D) Re-indexing / Index reorganisation

■ **Correct Answer: (D)**

Explanation: *Re-indexing drops old indexes and rebuilds them from scratch. Done after bulk updates or when index fragmentation significantly degrades retrieval performance.*

Q202. An ADVANTAGE of indexing is:

A) Eliminates the need for sorting
B) Works only with hashed files
C) Speeds up retrieval without scanning all records
D) Requires no extra disk storage

■ **Correct Answer: (C)**

Explanation: *The index allows the system to find any record using a fast lookup (binary search or tree traversal) instead of a slow O(N) sequential scan.*

Q203. A DISADVANTAGE of indexing is:

A) Cannot support fast retrieval
B) Requires a hash function
C) Cannot support sorted output
D) Index must be updated on every insert, delete, or key-field update

■ **Correct Answer: (D)**

Explanation: *Every write operation must also update the index to keep it consistent. For write-heavy files, this overhead is a significant disadvantage.*

Q204. True/False: A secondary index requires the data file to be physically sorted.

A) True only for large files
B) True only for binary files
C) False — a secondary index can exist on an unsorted or differently-ordered data file
D) True

■ **Correct Answer: (C)**

Explanation: *Secondary indexes are non-clustered by nature — they can be built on any attribute regardless of the physical sort order of the data file.*

Q205. Assertion (A): An index increases both retrieval speed AND storage requirements. Reason (R): The index structure stores additional key-address pairs, occupying extra disk space.

A) A is true; R is false
B) Both are false
C) A is false; R is true
D) Both A and R are true, and R correctly explains A

■ **Correct Answer: (D)**

Explanation: *Indexes speed up reads (A) by storing extra (key, address) pairs (R) which occupy additional disk space. Both true; R explains A.*

Q206. Match: Index Type → Defining Characteristic 1-Dense 2-Sparse 3-Secondary A-Based on non-primary attribute B-One entry per data block C-One entry per data record

A) 1-A, 2-C, 3-B
B) 1-C, 2-A, 3-B
C) 1-B, 2-A, 3-C
D) 1-C, 2-B, 3-A

■ **Correct Answer: (D)**

Explanation: *Dense: one entry per record (C). Sparse: one entry per block (B). Secondary: built on non-primary attribute (A).*

Q207. The INVERTED INDEX maps:

A) Attribute values to lists of records containing that value
B) Hash values to buckets
C) Line numbers to records
D) Record addresses to key values

■ **Correct Answer: (A)**

Explanation: *An inverted index maps each attribute value (e.g., a word, a category) to a list of record IDs that contain it. Used in full-text search and information retrieval.*

Q208. File UPDATING refers to:

A) Merging two files into one
B) Building an index on the file
C) Making changes to a file to keep it current — inserting, modifying, or deleting records
D) Sorting records in a file

■ **Correct Answer: (C)**

Explanation: *Updating encompasses all operations that change file content: INSERT (add new record), MODIFY (change field values), DELETE (remove record).*

Q209. Which is NOT a basic file update operation?

A) Modification of a field value
B) Sorting all records by key
C) Insertion of a new record
D) Deletion of an existing record

■ **Correct Answer: (B)**

Explanation: *The three basic update operations are Insert, Delete, Modify. Sorting is a separate file handling function, not an update operation.*

Q210. BATCH UPDATING involves:

A) Collecting all transactions over a period and processing them all at once in one run
B) Indexing transactions as they are collected
C) Hashing each transaction as it arrives
D) Updating records one at a time as each transaction arrives in real time

■ **Correct Answer: (A)**

Explanation: *Batch updating accumulates transactions (e.g., all day's sales) and applies them to the master file in a single scheduled run (e.g., overnight). Efficient for high-volume periodic updates.*

Q211. ONLINE (REAL-TIME) UPDATING means:

A) Updates are collected and applied together in batches
B) Updates are applied immediately as each transaction occurs
C) Sorting is required before any update
D) Only hashed files can be updated online

■ **Correct Answer: (B)**

Explanation: *Real-time updating reflects changes immediately. Example: ATM withdrawal immediately reduces account balance — the master file is updated at transaction time.*

Q212. In sequential BATCH UPDATE, the transaction file and master file must both be:

A) Sorted on the SAME key field
B) In binary file format
C) Hashed
D) Indexed

■ **Correct Answer: (A)**

Explanation: *The batch sequential update algorithm works by comparing records from both files key-by-key in order. Both files must be sorted on the same key for this to work correctly.*

Q213. When a record is LOGICALLY DELETED:

A) The file is re-sorted automatically
B) It is physically removed from disk immediately
C) The index is rebuilt immediately
D) It is marked as deleted (e.g., a flag bit) but space is not immediately reclaimed

■ **Correct Answer: (D)**

Explanation: *Logical deletion marks a record as invalid without removing it. Space is reclaimed later during file reorganisation. This avoids costly immediate restructuring.*

Q214. When a record is PHYSICALLY DELETED:

A) The file size stays the same
B) The record is moved to an overflow area
C) The record is actually removed and storage space can be reclaimed
D) Only a deletion flag is set

■ **Correct Answer: (C)**

Explanation: *Physical deletion actually removes the record bytes from the file. The freed space can be reused for future insertions or reclaimed by compaction.*

Q215. INSERTION into a SEQUENTIAL file typically requires:

A) Reading and rewriting part or all of the file to accommodate the new record
B) Direct insertion at the correct position without affecting other records
C) Applying a hash function
D) No changes to any other records

■ **Correct Answer: (A)**

Explanation: *Sequential files have no built-in free gaps. Inserting mid-file requires shifting subsequent records or rewriting the file to make room for the new record.*

Q216. Which updating strategy is used by ATM systems?

A) Weekly sequential update
B) Monthly merge
C) Real-time online updating
D) Batch updating (nightly)

■ **Correct Answer: (C)**

Explanation: *ATM systems must update balances immediately — any delay could allow overdraft through simultaneous transactions. Batch updating would leave records stale between runs.*

Q217. An ADVANTAGE of BATCH UPDATING is:

A) No transaction file required
B) Efficient processing of large transaction volumes in one scheduled run
C) No need for sorted input files
D) Immediate reflection of all changes

■ **Correct Answer: (B)**

Explanation: *Batch updating efficiently handles large volumes: one sequential pass through both sorted files produces the updated master. Minimal per-transaction overhead.*

Q218. A DISADVANTAGE of BATCH UPDATING is:

A) The master file may be stale (not current) between batch runs
B) Requires hashing of all records
C) Too complex to implement
D) Cannot handle large transaction volumes

■ **Correct Answer: (A)**

Explanation: *Between batch runs, new transactions are queued but not yet applied. The master file doesn't reflect recent events until the next batch run completes.*

Q219. True/False: In batch sequential update, the transaction file must be sorted in the SAME KEY ORDER as the master file.

A) False
B) True — both must be sorted on the same key for sequential comparison to work
C) True only for indexed master files
D) True only for large files

■ **Correct Answer: (B)**

Explanation: *The sequential update algorithm advances two pointers (one per file) in key order. If keys don't match in sort order, the comparison fails and incorrect output results.*

Q220. Assertion (A): Batch updating suits monthly payroll processing. Reason (R): Payroll is processed periodically for all employees at once, not in real time.

A) Both are false
B) A is false; R is true
C) Both A and R are true, and R correctly explains A
D) A is true; R is false

■ **Correct Answer: (C)**

Explanation: *Payroll is classic batch — all employees processed at month end (A). The periodical all-at-once nature makes batch the right approach (R). Both true; R explains A.*

Q221. Assertion (A): Real-time updating is essential for stock exchange systems. Reason (R): Stock prices change continuously and must be reflected immediately.

A) Both are false
B) A is true; R is false
C) A is false; R is true
D) Both A and R are true, and R correctly explains A

■ **Correct Answer: (D)**

Explanation: *Stock trading needs immediate price reflection (A). Prices change every second making batch unsuitable (R). Both true; R explains A.*

Q222. Which update operation INCREASES the number of records in a file?

A) Logical deletion flag
B) Insertion
C) Modification of a field
D) Deletion

■ **Correct Answer: (B)**

Explanation: *Only insertion adds a new record. Deletion reduces the count; modification changes existing records without changing the count.*

Q223. In a HASHED FILE, updating a record whose KEY FIELD has changed requires:

A) Deleting the old record and inserting a new one at the new hash address
B) Only modifying the field value in place at the same bucket
C) Re-sorting the file
D) Rebuilding the entire index

■ **Correct Answer: (A)**

Explanation: *Since the hash address depends on the key, a key change means the record must move to a different bucket. Old record deleted; new record inserted at h(new_key).*

Q224. FILE COMPACTION (reorganisation) is performed to:

A) Build a new index
B) Reclaim space from logically deleted records and restore file performance
C) Sort the file
D) Convert the file to hashed format

■ **Correct Answer: (B)**

Explanation: *Over time, logical deletions leave wasted gaps. Compaction reads only valid records and rewrites them contiguously, eliminating gaps and shrinking the file.*

Q225. In a sequential batch update: Transaction key < Master key means:

A) Apply the transaction to the master record
B) This is a possible error — transaction has no matching master record
C) Write the master record unchanged to the new master file
D) End of file reached

■ **Correct Answer: (B)**

Explanation: *If a DELETE or MODIFY transaction's key is smaller than the current master key, no matching master record was found. This orphan transaction is an error — logged and skipped.*

Q226. In a sequential batch update: Transaction key = Master key means:

A) Insert a new index entry
B) Skip both records
C) Apply the transaction (update or delete) to the master record
D) Create a new hash value

■ **Correct Answer: (C)**

Explanation: *A matching key means a transaction exists for this master record. Apply the modification or deletion, and write the result to the new master file.*

Q227. In a sequential batch update: Transaction key > Master key means:

A) Create a new index
B) Copy the master record unchanged to the new master file (no transaction for it)
C) Apply the transaction to the master record
D) Delete the master record

■ **Correct Answer: (B)**

Explanation: *No transaction exists for this master record. Copy it as-is to the new master. Advance the master file pointer to the next record.*

Q228. True/False: Online updating is generally MORE COMPLEX and COSTLY than batch updating.

A) False
B) True only for large files
C) True only for indexed files
D) True — concurrency control, immediate consistency, and 24/7 availability add complexity

■ **Correct Answer: (D)**

Explanation: *Real-time systems require record locking, immediate backup, concurrency control, and continuous availability — all significantly more complex than simple batch systems.*

Q229. Which update operation DECREASES the number of records in a file?

A) Indexing
B) Modification
C) Insertion
D) Deletion

■ **Correct Answer: (D)**

Explanation: *Only deletion removes a record. Insertion increases count; modification changes existing records without affecting count.*

Q230. True/False: After a batch update, the OLD master file is kept as a BACKUP.

A) True only for indexed master files
B) True — it becomes the 'father' file in the backup rotation
C) False — it is always immediately deleted
D) True only for large organisations

■ **Correct Answer: (B)**

Explanation: *After a batch update, the old master (father) is retained. If the new master (son) is corrupted, the father + latest transaction file can recreate it.*

Q231. A TRANSACTION LOG during file updating records:

A) Hash collisions
B) Index overflow entries
C) Sorted run boundaries in external sort
D) All changes made to the master file for accountability and recovery

■ **Correct Answer: (D)**

Explanation: *A transaction log captures every change: what was changed, when, and by whom. It enables audit trails and allows recovery by replaying transactions after a failure.*

Q232. File 'PURGING' means:

A) Sorting the file by a new key
B) Converting the file to hashed organisation
C) Permanently removing logically-deleted or obsolete records from the file
D) Building a new index

■ **Correct Answer: (C)**

Explanation: *Purging physically removes records that are no longer needed (old transactions, logically deleted records) to reduce file size and improve future performance.*

Q233. Which of the following correctly describes the ROLE OF INDEXING in file updating?

A) Indexing is unaffected by updates
B) Indexing slows down all update operations
C) The index must be updated whenever records are inserted, deleted, or key fields are modified
D) Indexing prevents records from being deleted

■ **Correct Answer: (C)**

Explanation: *Indexes must always be consistent with the data file. Any change affecting an indexed key requires a corresponding index update.*

Q234. True/False: Merging always requires the two input files to have the same NUMBER OF RECORDS.

A) True only for 2-way merge
B) False — input files can have different numbers of records
C) True only for text files
D) True

■ **Correct Answer: (B)**

Explanation: *Merging does not require equal-sized inputs. Files of any sizes can be merged — output contains all records from all inputs.*

Q235. The CORRECT SEQUENCE in a batch update cycle is:

A) Index → Hash → Sort → Merge
B) Update → Sort → Merge
C) Collect transactions → Sort transactions → Merge with master → New master output
D) Merge → Sort → Update

■ **Correct Answer: (C)**

Explanation: *Standard batch cycle: (1) collect transactions, (2) sort transaction file by key, (3) sequential merge of sorted transactions with sorted master file, (4) new master produced.*

Q236. True/False: The SORT KEY and MERGE KEY must be the same field in a sort-merge process.

A) False
B) True — if they differ, the merge comparison is on a different order than the sort, producing incorrect output
C) True only for large files
D) True only for binary files

■ **Correct Answer: (B)**

Explanation: *If the sort key and merge key differ, the merge algorithm compares on one field while records are ordered by another — producing unsorted, incorrect output.*

Q237. The MAIN ADVANTAGE of N-WAY MERGE over 2-way merge is:

A) Fewer merge passes needed — higher merge order means fewer iterations
B) No need for sorted input
C) Simpler implementation
D) Uses less memory during merging

■ **Correct Answer: (A)**

Explanation: *With k-way merge: passes = ■log_k(R)■. Larger k reduces the number of passes needed to merge R initial runs. Fewer passes = less total disk I/O.*

Q238. True/False: SORTING is a prerequisite for efficient file MERGING.

A) True only for binary files
B) True — both input files must be sorted on the same key for merging to produce a sorted output
C) False
D) True only for external files

■ **Correct Answer: (B)**

Explanation: *Merging only produces a correctly sorted output when both inputs are sorted on the same key. Sorting is the mandatory prerequisite.*

Q239. A REAL-WORLD analogy for a file INDEX is:

A) Playing a cassette tape from start to end
B) Throwing darts randomly at a target
C) Reading a novel from page 1 to last page
D) Using the index at the back of a textbook to find a topic by page number

■ **Correct Answer: (D)**

Explanation: *A textbook index maps topics (keys) to page numbers (disk addresses), allowing fast topic lookup without reading the whole book — exactly like a file index.*

Q240. Which of the four file handling functions BEST describes the process of keeping a file ACCURATE and CURRENT?

A) Sorting
B) Merging
C) Indexing
D) Updating

■ **Correct Answer: (D)**

Explanation: *Updating (inserting new records, modifying changed data, deleting obsolete records) is the function that keeps a file accurate, complete, and current over time.*

SECTION 5: COMPREHENSIVE, COMPARISON, ASSERTION-REASON & SCENARIO MCQs

Q241. Which file handling function does NOT change the NUMBER OF RECORDS?

A) Insertion
B) Deletion
C) Sorting
D) Merging

■ **Correct Answer: (C)**

Explanation: *Sorting rearranges existing records but neither adds nor removes any. Insertion adds, deletion removes, merging adds all records from input files.*

Q242. Statement 1: Sorting enables binary search on a sequential file. Statement 2: Merging two sorted files always produces an unsorted output. Which is CORRECT?

A) Both statements are true
B) Statement 1 is TRUE; Statement 2 is FALSE
C) Both statements are false
D) Statement 1 is false; Statement 2 is true

■ **Correct Answer: (B)**

Explanation: *Sorting enables binary search (S1 TRUE). Merging two sorted files produces a SORTED output — not unsorted (S2 FALSE).*

Q243. Statement 1: Indexed files require more storage than pure sequential files. Statement 2: Hashed files efficiently support range queries. Which is CORRECT?

A) Both are true
B) Both are false
C) Statement 1 is TRUE; Statement 2 is FALSE
D) Statement 1 is false; Statement 2 is true

■ **Correct Answer: (C)**

Explanation: *Indexed files carry index overhead (S1 TRUE). Hashed files are POOR for range queries because hashing destroys key order (S2 FALSE).*

Q244. Statement 1: Line sequential files are stored in text format. Statement 2: Hashed files are always stored in text format. Which is CORRECT?

A) Both false
B) Statement 1 is false; Statement 2 is true
C) Both true
D) Statement 1 is TRUE; Statement 2 is FALSE

■ **Correct Answer: (D)**

Explanation: *Line sequential files ARE text files (S1 TRUE). Hashed files are typically stored as binary files (S2 FALSE).*

Q245. Match: File Handling Function → Primary Purpose 1-Sorting 2-Merging 3-Indexing 4-Updating A-Keep file current B-Speed up retrieval C-Combine sorted files D-Arrange in order

A) 1-A, 2-B, 3-C, 4-D
B) 1-C, 2-A, 3-D, 4-B
C) 1-D, 2-C, 3-B, 4-A
D) 1-B, 2-D, 3-A, 4-C

■ **Correct Answer: (C)**

Explanation: *Sorting: arrange in order (D). Merging: combine sorted files (C). Indexing: speed up retrieval (B). Updating: keep file current (A).*

Q246. Match: File Organisation → Key Feature 1-Sequential 2-Indexed 3-Hashed 4-Line Sequential A-Newline delimiters B-Hash function C-Index table D-Sorted physical storage

A) 1-C, 2-A, 3-D, 4-B
B) 1-B, 2-D, 3-A, 4-C
C) 1-D, 2-C, 3-B, 4-A
D) 1-A, 2-B, 3-C, 4-D

■ **Correct Answer: (C)**

Explanation: *Sequential: sorted physical storage (D). Indexed: index table (C). Hashed: hash function (B). Line sequential: newline delimiters (A).*

Q247. Scenario: A hospital needs both 'instant patient record by ID' AND 'sorted patient lists per doctor'. The BEST organisation is:

A) Pure hashed
B) Line sequential
C) Pure sequential
D) Indexed sequential (ISAM/VSAM)

■ **Correct Answer: (D)**

Explanation: *ISAM/VSAM provides indexed access for instant patient-ID lookup AND sequential access for sorted report generation. No other single organisation serves both efficiently.*

Q248. Scenario: A company stores employee records in a hashed file. HR needs an alphabetically sorted list. Which function is needed?

A) Sorting (or building a sorted index)
B) Updating
C) Rehashing
D) Merging

■ **Correct Answer: (A)**

Explanation: *Hashed files have no inherent sort order. To produce a sorted list, the records must be physically sorted or accessed via a sorted secondary index.*

Q249. Scenario: Two branch offices have sorted customer files. Head office needs one sorted list. Required operation:

A) Index
B) Sort
C) Update
D) Merge

■ **Correct Answer: (D)**

Explanation: *Two pre-sorted files can be efficiently combined into one sorted file using the merge operation — no re-sorting of the entire dataset needed.*

Q250. Scenario: An ATM must update account balance IMMEDIATELY when a withdrawal occurs. This requires:

A) Weekly sequential update
B) Monthly merge
C) Batch updating
D) Real-time online updating

■ **Correct Answer: (D)**

Explanation: *ATM systems must update immediately — any delay risks overdraft through simultaneous transactions. Batch would leave balances stale.*

Q251. Assertion (A): External merge sort is used for files that CANNOT fit in RAM. Reason (R): Internal sorting algorithms like quicksort operate entirely in RAM.

A) A is false; R is true
B) Both are false
C) Both A and R are true, and R correctly explains A
D) A is true; R is false

■ **Correct Answer: (C)**

Explanation: *Files too large for RAM require external sort (A). Internal algorithms need all data in RAM (R). R explains why external sort is needed when file exceeds RAM.*

Q252. Assertion (A): An index always speeds up ALL types of queries equally. Reason (R): Indexes are most effective for equality and range queries, but may slow down full sequential scans.

A) A is FALSE; R is TRUE
B) Both A and R are true
C) Both are false
D) A is true; R is false

■ **Correct Answer: (A)**

Explanation: *Indexes don't help (can slow) full table scans since the index adds extra I/O. Assertion A is false; R is a true, nuanced observation.*

Q253. Assertion (A): In batch sequential update, a new master file is created — the old master is not modified in place. Reason (R): Sequential files cannot be efficiently updated in place.

A) A is true; R is false
B) A is false; R is true
C) Both A and R are true, and R correctly explains A
D) Both are false

■ **Correct Answer: (C)**

Explanation: *Sequential update always produces a new output file (A). In-place modification would require shifting records for insertions (R). Both true; R explains A.*

Q254. True/False: A file can have BOTH a primary index AND multiple secondary indexes.

A) False
B) True only for text files
C) True — one clustered primary index plus multiple non-clustered secondary indexes
D) True only for VSAM

■ **Correct Answer: (C)**

Explanation: *Yes. A file can have one clustered primary index (determining physical order) and multiple non-clustered secondary indexes on other attributes.*

Q255. True/False: The grandfather-father-son strategy applies ONLY to hashed files.

A) True only for large files
B) True
C) False — it applies to sequential (batch) file update systems in general
D) True only for text files

■ **Correct Answer: (C)**

Explanation: *The grandfather-father-son strategy was developed for magnetic-tape-based sequential file batch processing. It is not restricted to any file organisation type.*

Q256. The PROCESS OF REORGANISING a file to reclaim space from deleted records is:

A) Hashing
B) File reorganisation / compaction
C) Merging
D) Sorting

■ **Correct Answer: (B)**

Explanation: *Compaction reads all valid (non-deleted) records and rewrites them contiguously, eliminating gaps from deleted records and restoring performance.*

Q257. Which file handling concept is MOST relevant to disaster recovery?

A) Backup generation (grandfather-father-son scheme)
B) Merging
C) Indexing
D) Sorting

■ **Correct Answer: (A)**

Explanation: *The grandfather-father-son backup strategy (part of the update cycle) is the primary mechanism for recovering from hardware failures in file-based systems.*

Q258. For a SORTED file of 1,000 records with an index, binary search requires approximately:

A) 10 comparisons (log■1000 ≈ 10)
B) 1 comparison
C) 500 comparisons
D) 1,000 comparisons

■ **Correct Answer: (A)**

Explanation: *Binary search: log■(1000) ≈ 10 comparisons. Each step halves the remaining search space. Far better than sequential search's 500 average.*

Q259. For HASHED ACCESS with a perfect hash function and 1,000 records, retrieval requires:

A) 1 direct computation
B) 1,000 comparisons
C) 500 comparisons
D) 10 comparisons

■ **Correct Answer: (A)**

Explanation: *Perfect hashing maps each key to a unique address with zero collisions. One hash computation → one disk access → record found. O(1).*

Q260. Correct ranking of access methods for SINGLE-RECORD RETRIEVAL (fastest first):

A) Hashed O(1) → Indexed O(log N) → Sequential O(N)
B) Indexed → Hashed → Sequential
C) All three are equally fast
D) Sequential → Indexed → Hashed

■ **Correct Answer: (A)**

Explanation: *For individual record retrieval: Hashed O(1) is fastest, Indexed O(log N) is middle, Sequential O(N) is slowest.*

Q261. Scenario: A system needs to: (1) Retrieve any employee record by ID instantly, (2) Generate monthly sorted payroll report, (3) Support salary range queries. BEST organisation:

A) Pure sequential
B) Pure hashed
C) Indexed sequential with B+ tree
D) Line sequential only

■ **Correct Answer: (C)**

Explanation: *B+ tree indexed sequential: O(log N) direct access by ID (1), sorted sequential scan for payroll (2), range queries via linked leaf nodes (3). Only this serves all three.*

Q262. Which file handling function combines sorted files from multiple branches?

A) Sort
B) Index
C) Merge
D) Hash

■ **Correct Answer: (C)**

Explanation: *Merging combines multiple pre-sorted files into one sorted output. Pre-sorted branch files are merged into a single consolidated sorted file at head office.*

Q263. The SORT KEY and MERGE KEY in a sort-merge update cycle should be:

A) Different fields for variety
B) The SAME field so the merge produces correctly sorted output
C) Always the physical record number
D) Always numeric fields

■ **Correct Answer: (B)**

Explanation: *If sort key and merge key differ, the merge comparison is on a different field than the sort order → incorrect (unsorted) output.*

Q264. Which should you choose for RANGE QUERIES — indexing or hashing?

A) Indexing (sorted index supports efficient range traversal)
B) Hashing
C) Neither
D) Both equally

■ **Correct Answer: (A)**

Explanation: *Indexing is preferred for range queries. The sorted index allows: find start of range (binary search), traverse sequentially to range end. Hashing cannot do this.*

Q265. Which should you choose for PURE EQUALITY LOOKUPS needing maximum speed — indexing or hashing?

A) Hashing O(1)
B) Both equally fast
C) Sequential O(N)
D) Indexing O(log N)

■ **Correct Answer: (A)**

Explanation: *For pure equality searches (find record with key = X), hashing provides O(1) average access — faster than O(log N) indexed access.*

Q266. A library needs to find books by AUTHOR NAME (non-primary key) quickly. Which is needed?

A) Secondary index on author name
B) Primary index on book ID
C) No index needed
D) Hash index on title

■ **Correct Answer: (A)**

Explanation: *Author name is a non-primary attribute. A secondary index on author name enables fast retrieval of all books by a given author without scanning the entire catalog.*

Q267. After a batch sequential update, the OLD MASTER FILE is:

A) Merged with the transaction file
B) Converted to a hashed file
C) Always immediately deleted
D) Kept as the 'father' backup for recovery

■ **Correct Answer: (D)**

Explanation: *Old master → father (backup). If new master (son) is corrupted, father + latest transactions recreate it. Retention is a key part of the backup strategy.*

Q268. True/False: Merging always produces ONE output file from multiple inputs.

A) True — the standard merge operation produces one combined sorted output file
B) True only for 2-way merge
C) True only for N-way merge where N>2
D) False

■ **Correct Answer: (A)**

Explanation: *The purpose of merging is to combine multiple sorted files into one larger sorted file. The output is always one file.*

Q269. True/False: The INDEX FILE in indexed organisation is sorted by the key field.

A) False
B) True only for dense indexes
C) True — the index must be sorted for binary search to work efficiently on it
D) True only for primary indexes

■ **Correct Answer: (C)**

Explanation: *The index is sorted by key value so binary search finds any key in O(log N). An unsorted index would require sequential search — defeating its purpose.*

Q270. Which correctly identifies the FOUR file handling functions in the OAVS TGT CS syllabus?

A) Sorting, Merging, Indexing, Updating
B) Indexing, Hashing, Merging, Deleting
C) Sorting, Hashing, Indexing, Updating
D) Merging, Hashing, Sorting, Deleting

■ **Correct Answer: (A)**

Explanation: *The OAVS TGT CS syllabus specifically lists: Sorting, Merging, Indexing, Updating as the four file handling functions.*

Q271. Which correctly identifies the THREE file organisation techniques in the OAVS TGT CS syllabus?

A) Sequential, Binary, Hashed
B) Indexed, Line Sequential, Hashed
C) VSAM, ISAM, Hashed
D) Sequential, Indexed, Direct

■ **Correct Answer: (B)**

Explanation: *The OAVS TGT CS syllabus specifically mentions: Indexed, Line Sequential, and Hashed as the three file organisation techniques.*

Q272. Which correctly identifies the THREE file access techniques in the OAVS TGT CS syllabus?

A) Indexed, Sequential, Hashed
B) Direct, Hashed, ISAM
C) Sequential, Random, Binary
D) Line Sequential, Hashed, Binary

■ **Correct Answer: (A)**

Explanation: *The OAVS TGT CS syllabus specifically lists: Indexed access, Sequential access, and Hashed access as the three file accessing techniques.*

Q273. True/False: Hashed access can efficiently find all records with salary between 30,000 and 50,000.

A) False — hash functions do not preserve order; range queries require scanning all buckets
B) True
C) True for small files
D) True if a special range-hash function is used

■ **Correct Answer: (A)**

Explanation: *Hash functions scatter records across buckets with no sorted order. A salary range query would require visiting every bucket — O(N), no better than sequential search.*

Q274. Which REAL-WORLD ANALOGY best represents sequential file organisation?

A) Playing a cassette tape from start to end
B) Using a hash-based post-box system
C) Using a library card index to find a book
D) Jumping to any page of a book directly

■ **Correct Answer: (A)**

Explanation: *A cassette tape must be played sequentially — you cannot jump to a specific song without fast-forwarding through earlier content. This mirrors sequential file access.*

Q275. Which REAL-WORLD ANALOGY best represents indexed file organisation?

A) Using the index at the back of a textbook to find a topic quickly
B) Playing a cassette tape sequentially
C) Shuffling a deck of cards
D) Reading a novel from page 1 to last page

■ **Correct Answer: (A)**

Explanation: *A textbook index maps topics to page numbers — exactly like a file index maps keys to disk addresses. Find the address in the index, then jump directly to that page.*

Q276. Which REAL-WORLD ANALOGY best represents hashed file organisation?

A) Using a book's index
B) A pigeonhole/post-box system where each surname initial maps to a specific slot
C) Playing a vinyl record
D) Reading a textbook sequentially

■ **Correct Answer: (B)**

Explanation: *A pigeonhole mail system assigns surnames to specific slots by initial — like a hash function assigning keys to specific buckets. Quick to place and retrieve.*

Q277. True/False: A perfectly designed file system uses only ONE of the four file handling functions.

A) True
B) False — real file systems use all four functions in combination for complete file maintenance
C) True only for read-only files
D) True only for small files

■ **Correct Answer: (B)**

Explanation: *Complete file maintenance requires all four: sort (order), merge (combine), index (fast access), update (currency). They complement and enable each other.*

Q278. The CORRECT ORDER of the batch update cycle steps is:

A) Hash → Sort → Merge → Update
B) Collect transactions → Sort transactions → Merge with master → New master
C) Update → Sort → Merge → Index
D) Merge → Sort → Update → Index

■ **Correct Answer: (B)**

Explanation: *Standard batch cycle: (1) collect transactions, (2) sort transaction file by master key, (3) sequential merge produces new master, (4) re-index if needed.*

Q279. Which is NOT covered in the OAVS TGT CS 'File' topic syllabus?

A) Hashed file organisation
B) Sorting
C) Indexed access technique
D) Database normalisation and SQL queries

■ **Correct Answer: (D)**

Explanation: *The OAVS 'File' topic covers: concept of file, indexed/line sequential/hashed organisation, access techniques, sorting/merging/indexing/updating. SQL and normalisation are not included.*

Q280. Which BEST summarises the complete 'File' topic as per the OAVS TGT CS syllabus?

A) Files are temporary; only sequential access; hashing is always best

B) Files provide PERSISTENT STORAGE; organised as indexed/line sequential/hashed; accessed via sequential/indexed/hashed access; maintained by sorting, merging, indexing, and updating

C) Files and databases are identical in all respects

D) Files always use SQL for management

■ **Correct Answer: (B)**

Explanation: *The complete topic scope: concept of file → file organisation (indexed, line sequential, hashed) → access techniques (sequential, indexed, hashed) → handling functions (sort, merge, index, update).*